

Guia de Funções Java — Referência do Nível Básico

Referência rápida das principais classes e métodos usados no aprendizado. Para cada método: assinatura, o que retorna, exemplo e casos-limite/observações. Complemento das apostilas de fundamentos, seguindo a metodologia DevSuperior.

Sumário

1. `Math`
2. `Character`
3. `Random`
4. `String`
5. `Arrays`
6. `Scanner`
7. `Integer` (e wrappers)
8. `StringBuilder`
9. `Collections`
10. Tabela-resumo de casos-limite

1. `Math`

Classe de utilidades matemáticas. Todos os métodos são **estáticos** (chama-se `Math.metodo()`, sem instanciar). Import automático (`java.lang`).

`Math.sqrt(double a)`

Retorna: `double` — a raiz quadrada.

```
Math.sqrt(16);    // 4.0
Math.sqrt(2);     // 1.4142135623730951
```

Observações: raiz de número negativo retorna `NaN` (Not a Number), não lança erro. `Math.sqrt(-4)` → `NaN`.

`Math.pow(double base, double exp)`

Retorna: `double` — base elevada ao expoente.

```
Math.pow(2, 10);  // 1024.0
Math.pow(5, 2);   // 25.0
```

Observações: sempre retorna `double`, mesmo com resultado "redondo" (`25.0`, não `25`). Para usar como `int`, precisa de cast: `(int) Math.pow(5, 2)`.

`Math.abs(x)`

Retorna: o valor absoluto (módulo), no mesmo tipo do argumento (`int`, `double` ...).

```
Math.abs(-7);     // 7
Math.abs(-3.5);   // 3.5
```

`Math.round(double a)`

Retorna: `long` (para `double`) — arredonda ao inteiro mais próximo.

```
Math.round(2.5); // 3
Math.round(2.4); // 2
Math.round(2.6); // 3
```

Observações: arredonda 0.5 **para cima**. Retorna `long`, pode precisar de cast para `int`.

Math.ceil(double a) / Math.floor(double a)

Retorna: `double` — arredonda para cima (`ceil`) ou para baixo (`floor`).

```
Math.ceil(2.1); // 3.0 (sempre sobe)
Math.floor(2.9); // 2.0 (sempre desce)
```

Observações: retornam `double` (`3.0`, não `3`).

Math.max(a, b) / Math.min(a, b)

Retorna: o maior / menor entre **dois** valores.

```
Math.max(5, 10); // 10
Math.min(5, 10); // 5
```

Casos-limite: aceita **exatamente 2** argumentos. Para 3+, encadeie (`Math.max(a, Math.max(b, c))`) ou use um loop/vetor. Não existe `Math.max(a, b, c)`.

Math.random()

Retorna: `double` entre `0.0` (incluído) e `1.0` (excluído).

```
Math.random(); // ex: 0.734...
(int)(Math.random() * 100); // inteiro de 0 a 99
```

Observações: para geração numérica mais controlada, prefira a classe `Random`.

Math.PI

Constante `double` de π .

```
Math.PI; // 3.141592653589793
```

2. Character

Métodos **estáticos** para analisar/transformar um `char`. Import automático (`java.lang`).

Character.isLetter(char c)

Retorna: `boolean` — se é letra (de qualquer alfabeto, inclui acentuadas).

```
Character.isLetter('A'); // true
Character.isLetter('ç'); // true
Character.isLetter('5'); // false
```

Character.isDigit(char c)

Retorna: `boolean` — se é dígito 0-9.

```
Character.isDigit('7'); // true
Character.isDigit('a'); // false
```

Character.isLetterOrDigit(char c)

Retorna: `boolean` — se é letra **ou** dígito.

```
Character.isLetterOrDigit('A'); // true
Character.isLetterOrDigit('@'); // false
```

Character.isWhitespace(char c)

Retorna: `boolean` — se é espaço, tab ou quebra de linha.

```
Character.isWhitespace(' '); // true
Character.isWhitespace('a'); // false
```

Character.isUpperCase / isLowerCase(char c)

Retorna: `boolean` — se é maiúscula / minúscula.

```
Character.isUpperCase('A'); // true
Character.isLowerCase('a'); // true
```

Character.toUpperCase / toLowerCase(char c)

Retorna: `char` — a versão maiúscula / minúscula.

```
Character.toUpperCase('a'); // 'A'
Character.toLowerCase('A'); // 'a'
```

Observações: opera em um `char`. Para strings inteiras, use `String.toUpperCase()`.

Padrão de uso combinado:

```
char c = texto.charAt(i);
if (Character.isLetter(c)) { ... }
```

3. Random

Gerador de números pseudoaleatórios. **Precisa instanciar** (`new Random()`). Import: `java.util.Random`.

new Random() / new Random(long seed)

```
Random r = new Random(); // sequência diferente a cada execução
Random r = new Random(42); // seed fixa → sequência sempre igual (reprodutível)
```

Observações: seed fixa é essencial para testes reprodutíveis (mesmo conceito de `random_state` em ML).

nextInt(int limite)

Retorna: `int` de 0 até `limite - 1` (limite **exclusivo**).

```
r.nextInt(100); // 0 a 99
r.nextInt(6) + 1; // 1 a 6 (dado)
```

Casos-limite: para incluir o 100, use `nextInt(101)`. Faixa arbitrária: `nextInt(max - min + 1) + min`.

nextInt()

Retorna: `int` qualquer (inclusive negativos grandes).

```
r.nextInt(); // ex: -1547203912
```

nextDouble()

Retorna: `double` entre `0.0` e `1.0`.

```
r.nextDouble(); // ex: 0.4521...
```

nextBoolean()

Retorna: `boolean` aleatório.

```
r.nextBoolean(); // true ou false
```

Observação geral: `Random` **nunca** tem problema de buffer (não lê de entrada, gera internamente).

4. String

Sequência **imutável** de caracteres (cada "modificação" cria uma nova String). Tipo referência.

length()

Retorna: `int` — quantidade de caracteres. **Com** parênteses (é método).

```
"Java".length(); // 4
```

Cuidado: em array é `.length` (atributo, sem parênteses); em String é `.length()` (método).

charAt(int index)

Retorna: `char` — caractere na posição (índice começa em `0`).

```
"Java".charAt(0); // 'J'
```

Casos-limite: índice fora do intervalo → `StringIndexOutOfBoundsException`. `charAt(0)` em string vazia estoura — valide `length()` antes.

equals(String outra) / equalsIgnoreCase(String outra)

Retorna: `boolean` — compara **conteúdo** (equals) / ignorando maiúsc./minúsc.

```
"abc".equals("abc"); // true
"abc".equalsIgnoreCase("ABC"); // true
```

Cuidado: para comparar Strings use `.equals()`, **nunca** `==` (que compara referência de memória).

contains(CharSequence s)

Retorna: `boolean` — se contém a substring.

```
"Java".contains("av"); // true
```

Casos-limite: `contains("")` sempre retorna `true` (toda string contém a vazia). Não aceita `char` — só `String`; para `char`, use `indexOf`.

indexOf(...) / lastIndexOf(...)

Retorna: `int` — posição da primeira / última ocorrência, ou `-1` se não existe.

```
"banana".indexOf('a');    // 1
"banana".indexOf('a', 2); // 3 (a partir da posição 2)
"banana".lastIndexOf('a'); // 5
"banana".indexOf('z');    // -1
```

Observações: aceita `char` e `String`. O `-1` é o sentinela universal de "não encontrado".

substring(int inicio) / substring(int inicio, int fim)

Retorna: `String` — trecho do índice início até fim (**fim exclusivo**).

```
"Programacao".substring(0, 4);    // "Prog"
"Programacao".substring(4);       // "amacao"
```

toUpperCase() / toLowerCase()

Retorna: `String` — nova string em maiúsc./minúsc.

```
"java".toUpperCase();    // "JAVA"
```

trim()

Retorna: `String` — sem espaços nas bordas.

```
"  oi  ".trim();    // "oi"
```

isEmpty()

Retorna: `boolean` — se é `""` (comprimento zero).

```
"".isEmpty();    // true
" ".isEmpty();   // false (tem um espaço)
```

replace(char velho, char novo)

Retorna: `String` — nova string com as substituições.

```
"casa".replace('a', '@');    // "c@s@"
```

split(String regex)

Retorna: `String[]` — divide pelo separador.

```
"a,b,c".split(",");    // ["a", "b", "c"]
```

repeat(int n)

Retorna: `String` — repete a string n vezes (Java 11+).

```
"=".repeat(5);    // "=====
```

toCharArray()

Retorna: `char[]` — os caracteres como array.

```
"abc".toCharArray();    // ['a', 'b', 'c']
```

Observação sobre imutabilidade: concatenar com `+=` em loops grandes é ineficiente (cada `+=` cria nova `String`). Para montagem intensa, use `StringBuilder` (seção 8).

5. Arrays

Utilitários **estáticos** para arrays. Import: `java.util.Arrays`.

Arrays.toString(array)

Retorna: `String` — representação legível de array 1D.

```
Arrays.toString(new int[]{1,2,3}); // "[1, 2, 3]"
```

Arrays.deepToString(array[][])

Retorna: `String` — representação de array 2D (matriz).

```
Arrays.deepToString(new int[][]{{1,2},{3,4}}); // "[[1, 2], [3, 4]]"
```

Casos-limite: para matriz, `toString` simples imprimiria endereços de memória — use `deepToString`.

Arrays.sort(array)

Retorna: `void` — ordena **em ordem crescente**, modificando o próprio array.

```
int[] v = {3, 1, 2};  
Arrays.sort(v); // v vira {1, 2, 3}
```

Casos-limite: altera o array original (não retorna cópia). Ordena só crescente por padrão.

Arrays.fill(array, valor)

Retorna: `void` — preenche todo o array com um valor.

```
int[] v = new int[3];  
Arrays.fill(v, 7); // {7, 7, 7}
```

Arrays.copyOf(array, novoTamanho)

Retorna: novo array (cópia independente).

```
int[] b = Arrays.copyOf(a, a.length);
```

Observações: cria cópia independente (diferente de `int[] b = a;`, que compartilha referência).

Arrays.equals(a, b)

Retorna: `boolean` — se os arrays têm os mesmos elementos.

```
Arrays.equals(new int[]{1,2}, new int[]{1,2}); // true
```

Cuidado: use isto, não `==` (que compara referências, não conteúdo).

6. Scanner

Leitura da entrada (teclado). **Precisa instanciar.** Import: `java.util.Scanner`.

new Scanner(System.in)

```
Scanner sc = new Scanner(System.in);
```

nextInt() / nextDouble()

Retorna: o número lido.

```
int n = sc.nextInt();
double d = sc.nextDouble();
```

Casos-limite: se o usuário digitar texto, lança `InputMismatchException`. Deixam a quebra de linha no buffer (ver `nextLine`).

`nextLine()`

Retorna: `String` — a linha inteira (incluindo espaços).

```
String linha = sc.nextLine();
```

Casos-limite (buffer): após `nextInt()` / `nextDouble()`, um `nextLine()` captura o `"\n"` residual (string vazia). Solução: um `sc.nextLine()` de limpeza entre eles.

`next()`

Retorna: `String` — só até o próximo espaço (uma "palavra").

```
String palavra = sc.next();
```

`useLocale(Locale) / close()`

```
Locale.setDefault(Locale.US); // define ponto decimal (fazer ANTES de criar o Scanner)
sc.close();                    // libera o recurso ao final
```

Observações: o Scanner captura o Locale no momento da criação — configure o Locale antes.

7. Integer (e wrappers)

`Integer` é o **wrapper** (objeto) do primitivo `int`. Existem equivalentes para todos: `Double`, `Character`, `Boolean`, etc. Import automático (`java.lang`).

`Integer.parseInt(String s)`

Retorna: `int` — converte texto em número.

```
Integer.parseInt("42"); // 42
```

Casos-limite: se a string não for um número válido, lança `NumberFormatException`. Trate com try/catch para validar entrada:

```
try {
    int n = Integer.parseInt(entrada);
} catch (NumberFormatException e) {
    System.out.println("Não é um número válido.");
}
```

`Integer.valueOf(String s)`

Retorna: `Integer` (objeto) — similar a `parseInt`, mas devolve o wrapper.

```
Integer.valueOf("42"); // Integer 42
```

`Integer.MAX_VALUE / MIN_VALUE`

Constantes com os limites do `int`.

```
Integer.MAX_VALUE;    // 2147483647
Integer.MIN_VALUE;    // -2147483648
```

Observações: útil para inicializar buscas de mínimo/máximo (e para entender overflow — ultrapassar o MAX "vira" para negativo).

Integer.toString(int i)

Retorna: `String` — o número como texto.

```
Integer.toString(42);    // "42"
```

Autoboxing / Unboxing

Java converte automaticamente entre `int` e `Integer`:

```
Integer obj = 5;         // autoboxing (int → Integer)
int x = obj;             // unboxing (Integer → int)
```

Observações: coleções (`List`, `Map`) só aceitam objetos (`Integer`), não primitivos (`int`) — por isso os wrappers existem. `Integer` pode ser `null`; `int` nunca (é 0 por padrão).

8. StringBuilder

String **mutável** — para montar/modificar texto com eficiência. Import automático (`java.lang`).

new StringBuilder()

```
StringBuilder sb = new StringBuilder();
```

append(...)

Retorna: o próprio `StringBuilder` (permite encadear).

```
sb.append("Olá");
sb.append(", ").append("mundo");    // encadeado
```

toString()

Retorna: `String` — converte o resultado final em String.

```
String resultado = sb.toString();
```

insert(int pos, valor) / reverse() / delete(int ini, int fim)

```
sb.insert(0, "X");    // insere na posição 0
sb.reverse();         // inverte o conteúdo
sb.length();          // tamanho atual
```

Por que usar (insight): `String` é imutável — concatenar com `+=` em loop cria um objeto novo a cada volta (lento em escala). `StringBuilder` altera o mesmo objeto:

```
// ✗ ineficiente em loops grandes
String s = "";
for (int i = 0; i < 10000; i++) s += i;

// ✓ eficiente
StringBuilder sb = new StringBuilder();
for (int i = 0; i < 10000; i++) sb.append(i);
String s = sb.toString();
```

9. Collections

Utilitários **estáticos** para coleções (`List`, `Set` ...). Import: `java.util.Collections`. Trabalha **apenas com objetos** (`Integer`, não `int`).

⚠ **Nível intermediário.** Requer conhecer `List` / `ArrayList` primeiro. Incluído aqui como referência antecipada.

`Collections.max(coleção)` / `Collections.min(coleção)`

Retorna: o maior / menor elemento.

```
List<Integer> lista = new ArrayList<>(List.of(3, 7, 1));
Collections.max(lista);    // 7
Collections.min(lista);    // 1
```

Casos-limite: lança exceção se a coleção estiver vazia. Dá o **valor**, não a posição.

`Collections.sort(lista)`

Retorna: `void` — ordena a lista em ordem crescente (modifica a original).

```
Collections.sort(lista);    // ordena no lugar
```

`Collections.reverse(lista)`

Retorna: `void` — inverte a ordem dos elementos.

```
Collections.reverse(lista);
```

Casos-limite (pegadinha clássica): `Collections.reverse(Arrays.asList(vetor))` **não** funciona com `int[]` (primitivo) — `asList` trata o array como um único elemento. Funciona com `Integer[]` (objetos).

`Collections.shuffle(lista)`

Retorna: `void` — embaralha aleatoriamente.

```
Collections.shuffle(lista);
```

`Collections.frequency(coleção, valor)`

Retorna: `int` — quantas vezes o valor aparece.

```
Collections.frequency(lista, 7);    // nº de ocorrências de 7
```

10. Tabela-resumo de casos-limite

Método	Armadilha	Como evitar
<code>Math.pow</code>	sempre retorna <code>double</code>	cast <code>(int)</code> se precisar de inteiro
<code>Math.max/min</code>	só 2 argumentos	encadear ou usar loop
<code>Math.sqrt(negativo)</code>	retorna <code>NaN</code> , não erro	validar sinal antes
<code>String.contains("")</code>	sempre <code>true</code>	validar string vazia
<code>String.contains(char)</code>	não compila	usar <code>indexOf(char)</code>
<code>String == String</code>	compara referência	usar <code>.equals()</code>
<code>charAt</code> em vazio	<code>StringIndexOutOfBoundsException</code>	checar <code>length()</code> antes
<code>nextInt</code> → <code>nextLine</code>	buffer com <code>"\n"</code> órfão	<code>sc.nextLine()</code> de limpeza
<code>nextInt</code> com texto	<code>InputMismatchException</code>	ler String + <code>parseInt</code> + try/catch
<code>Integer.parseInt</code> inválido	<code>NumberFormatException</code>	envolver em try/catch
<code>Arrays.sort</code>	modifica o original	copiar antes se precisar do original
<code>int[] b = a</code>	compartilha referência	usar <code>.clone()</code> ou <code>Arrays.copyOf</code>
<code>Collections.reverse</code> + <code>int[]</code>	não inverte	usar <code>Integer[]</code>
<code>Collections.max</code> (vazia)	lança exceção	checar se não está vazia
ultrapassar <code>Integer.MAX_VALUE</code>	overflow (vira negativo)	usar <code>long</code> se necessário

Guia em evolução — atualizado conforme o avanço nos estudos.

Jader Silva · GitHub: MONTANHA702 · LinkedIn: /in/jaderssantos/